Adaptive Policies: Code Policies for Embodied Agents

Siphesihle Mthethwa
Supervised by Dr. Steven James
School of Computer Science and Applied Mathematics
University of the Witwatersrand
Johannesburg, South Africa
2093411@students.wits.ac.za
https://adaptivepolicies.github.io/

Abstract

Code policies, which are Python programs that control an agent's actions, may fail when their assumptions do not match the environment. Repairing such failures requires execution feedback, while repeated model invocations increase inference cost. We introduce Adaptive Policies, which extend generated Python policies with checkpoints that trigger model-assisted repair when execution fails. Repair escalates from local recovery to helper regeneration and task re-planning. We evaluate Adaptive Policies against ProgPrompt and Code as Policies (CaP) in the VirtualHome household simulator using two open-weight language models, Qwen3.5-9B and Gemma 4 12B. Adaptive Policies achieves task success rates of 24.3% and 24.6%, compared with 13.0% and 8.9% for our CaP baseline. However, it does not consistently outperform ProgPrompt and requires more model invocations. Enabling extended reasoning does not consistently improve success under the evaluated output budget and substantially increases generation time. These results show that execution-time repair improves the success of generated code policies, but its benefits depend on producing executable code and come at an additional inference cost.

1 Introduction

Planning lets embodied agents, such as household robots, devise sequences of actions that achieve a goal in a physical or simulated environment. Large Language Models (LLMs) are attractive planners for such agents because they interpret natural-language instructions and carry broad commonsense knowledge about everyday objects and activities. However, generating a plausible plan does not ensure that its actions are executable or that they achieve the goal (Kambhampati et al. 2024; Valmeekam et al. 2023a). An agent must account for action preconditions and changes in the environment during execution.

Program-based approaches give the model an interface for expressing actions and interacting with the environment. ProgPrompt supplies action imports, an object list and example programs, and combines generated plans with assertion-based recovery steps (Singh et al. 2023). Code as Policies generates Python programs that compose perception and control functions using conditions, loops and helper functions (Liang et al. 2023). Whereas an action-sequence plan specifies steps to perform, a *code policy* can use observations to decide which actions to perform during execution. This flexibility does not by itself ensure that a generated policy can recover when an action fails or an expected condition does not hold.

Execution feedback provides a way to address such failures. Existing approaches use observations to re-plan (Song et al. 2023; Huang et al. 2022b; Yao et al. 2023b) or combine generation with external verification, as in the LLM-Modulo framework (Kambhampati et al. 2024; Gundawar et al. 2024). Model-assisted repair, however, requires additional inference. This raises the question addressed here: when a generated code policy fails, how can execution feedback guide repair, and what does that repair cost?

We propose *Adaptive Policies*, an extension of the code-policy approach in Code as Policies. Generated policies check expected conditions during execution through *adaptive checkpoints*. Passing a checkpoint requires no model call; a failed checkpoint, action or code execution can trigger model-assisted repair using the current observation and failure context. Repair escalates from local recovery to helper regeneration and task re-planning (Figure 1). The aim is to use model-assisted repair when execution deviates from expectations while allowing successful execution to continue without a repair call.

We evaluate Adaptive Policies against ProgPrompt and our adaptation of Code as Policies in the VirtualHome household simulator (Puig et al. 2018). We also examine whether extended reasoning before code generation improves task success enough to justify its inference cost. Reasoning language models are trained to produce extended reasoning traces before answering (Guo et al. 2025); here, we compare reasoning enabled and disabled within the same model weights using two open-weight models. Adaptive Policies improves task success over our Code as Policies baseline, but its performance relative to ProgPrompt is mixed and recovery requires more model invocations. Under the evaluated output budget, enabling reasoning substantially increases generation time without consistently improving task success.

Our contributions are:

- **Adaptive Policies**, which extends generated code policies with execution checks and model-assisted repair that

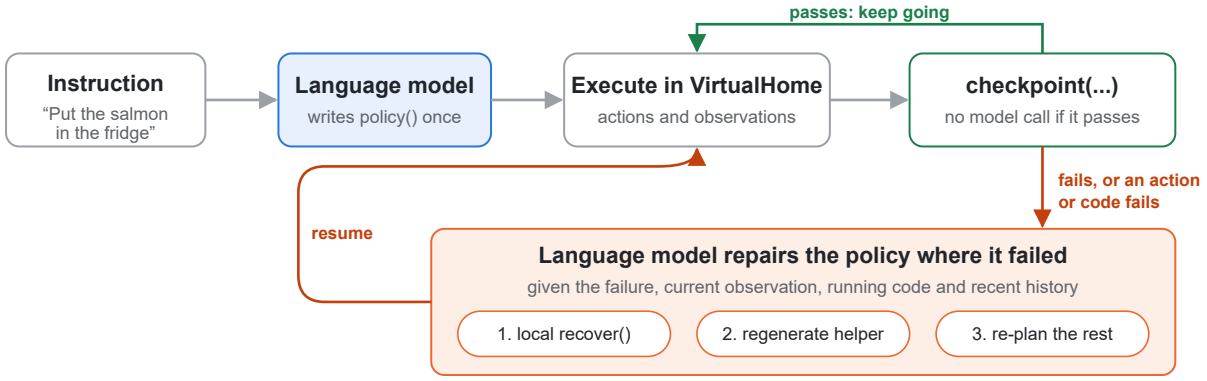


Figure 1: Adaptive policies. The language model writes a Python policy once. While it executes in VirtualHome, the policy's checkpoints test the states that later steps rely on; a checkpoint that passes costs no model call. When a checkpoint, an action or the generated code fails, the model is called with the failure, the agent's current observation, the running code and the recent history, and the repair escalates only as far as needed: a local `recover()`, then regenerating the failing helper, then re-planning the rest of the task. Execution then resumes from where it failed.

escalates from local recovery to helper regeneration and task re-planning;

- an **evaluation** against ProgPrompt and Code as Policies in VirtualHome across two models, reasoning settings and task suites, including three checkpoint placements for Adaptive Policies; and
- an **analysis of task success, inference cost and failure modes**, including the contribution of recovery compared with stopping at the first execution failure.

2 Background and Related Work

Planning and partial observability. Following state-transition and observation-history formulations of planning (LaValle 2006; Kaelbling, Littman, and Cassandra 1998), a goal-directed problem with deterministic transitions and partial observations can be written as

$$\mathcal{P} = (S, A, T, \Omega, O, s_0, G). \tag{1}$$

Here, S is the state space, A the action set, $s_0 \in S$ the initial state, and $G \subseteq S$ the goal states. The partial function $T : S \times A \rightharpoonup S$ specifies action effects where preconditions hold. The observation function $O : S \to \Omega$ gives $o_t = O(s_t)$; different states can produce the same observation, so s_t need not be known to the agent. An action-sequence plan $p = (a_0, \ldots, a_{n-1})$ solves the instance if every transition $s_{t+1} = T(s_t, a_t)$ is defined and $s_n \in G$. A policy instead selects actions from the available history:

$$a_t = \pi(h_t), \qquad h_t = (o_0, a_0, o_1, \ldots, a_{t-1}, o_t). \tag{2}$$

It may also terminate execution. This distinction allows policies to respond to new observations rather than follow a fixed sequence. In our study, observations are symbolic descriptions derived from the simulator graph, restricted to the agent's current room; the policy is not given the full state.

Generated plans and policies. LLM-generated plans need not satisfy action preconditions or achieve their goals (Valmeekam et al. 2023a,b). Grounding methods map language to available actions (Huang et al. 2022a), use learned affordances (Brohan et al. 2023; Hazra, Dos Martires, and De Raedt 2024), or delegate search to a classical planner after translating the task into a formal model (Liu et al. 2023). Planner guarantees apply to that model, rather than establishing that its translation matches the environment.

ProgPrompt generates Python-like plans with assertions and fixed recovery branches (Singh et al. 2023). Code as Policies generates Python control flow combining perception, actions and recursive helper functions (Liang et al. 2023). ReAct, Inner Monologue and LLM-Planner use observations for subsequent reasoning or re-planning (Yao et al. 2023b; Huang et al. 2022b; Song et al. 2023); LLM-Modulo uses external verification (Kambhampati et al. 2024). Our question concerns when execution should request model-assisted repair of a generated policy.

Reasoning and cost. Chain-of-thought prompting can produce reasoning within one response (Wei et al. 2022), whereas sampling and search can add invocations (Wang et al. 2023; Yao et al. 2023a; Besta et al. 2024). Reasoning models such as DeepSeek-R1 are trained to generate extended reasoning (Guo et al. 2025); o1 improved PlanBench results but remained sensitive to problem size and domain terminology (Valmeekam, Stechly, and Kambhampati 2024). We compare reasoning off/on within the same model weights and distinguish calls, tokens and request time. Extended related work is in Appendix A.

3 Research Problem

The objective is to achieve an embodied task under partial observability while maintaining action executability and limiting model invocations. Adaptive Policies investigates checkpoints and escalating repair as one approach to this tradeoff. Its evaluated comparison is with ProgPrompt and our Code as Policies adaptation; a dedicated per-step re-planning baseline is outside scope. We ask:

RQ1: Performance. How do reasoning enabled/disabled and the three AP checkpoint conditions affect task success, partial success and executability relative to the baselines?

RQ2: Cost. How do model calls, action counts, token use and generation time compare, and what cost accompanies recovery?

RQ3: Failures. Which observed failures involve truncation, invalid code, API errors, resource limits or unsuccessful checkpoint recovery?

The study classifies recorded outcomes rather than establishing every failure's cause through controlled interventions. It evaluates two models with symbolic partial observations in VirtualHome, without learned perception or controlled external perturbations.

4 Methods

4.1 Environment and task representation

All methods act in VirtualHome (Puig et al. 2018), where an agent executes discrete scripted actions (walk, find, grab, open, put-in, switch-on and so on) on objects in a 3D household scene. The environment state is a graph whose nodes are objects with states (e.g. `OPEN`, `ON`) and whose edges are relations (e.g. `INSIDE`, `ON`, `CLOSE`). The agent receives a partial graph of objects visible from its current room and their relations; objects hidden inside closed containers are excluded. This is symbolic visibility filtering rather than learned visual perception. Scene-level object names or classes and room names are supplied as prompt context, but do not reveal all current object states and locations. CaP and AP query current observations through the same API. Queries distinguish unknown container contents from an observed empty container, and actions accept object references only when the object is currently observed or held.

We use a custom build of the VirtualHome v2.3.0 Linux player. It keeps the original scenes, object identities and meshes. It also adds native support for interactions that the ProgPrompt reference programs require but the stock player rejected (for example, curtains, windows, the washing machine, rug folding, tool use and the kitchen door). Every action is first checked against symbolic preconditions and then executed in Unity; only a confirmed native success updates the symbolic state. A native rejection blocks further actions for ProgPrompt and CaP. AP instead permits continuation after a known rejection, so its recovery mechanism can respond; an uncertain native state remains terminal. This difference is part of the evaluated methods.

As reference validation, replaying all 69 reference programs (534 actions) in this build achieves SR, PSR, Exec and precision of 1. This establishes that these reference programs are executable and consistent with their goal graphs under the shared evaluator; it does not establish the correctness of arbitrary generated policies.

4.2 Baseline 1: ProgPrompt

ProgPrompt represents a plan as a Python-like function (Singh et al. 2023). The prompt contains:

```
def refrigerate_the_salmon():
  # 0: go to the kitchen and find the
      salmon
  find('kitchen')
  find('salmon')
  # 1: put the salmon in the fridge
  assert('close' to 'salmon')
    else: find('salmon')
  grab('salmon')
  find('fridge')
  assert('fridge' is 'closed')
    else: close('fridge')
  open('fridge')
  putin('salmon', 'fridge')
  # 2: close the fridge
  close('fridge')
  # 3: Done
```

Listing 1: An abridged ProgPrompt training program in its Python-like domain-specific language, rather than directly executable Python. Assertions are evaluated at run time; a false assertion runs the indented `else` actions.

1. an import line listing the 24 available actions and their arity;
2. the list of objects in the scene;
3. example task programs; and
4. the target function header, such as `def refrigerate_the_salmon():`.

The model completes the body. Programs contain numbered sub-task comments and *assertions* with recovery actions in a small domain-specific language (Listing 1).

A parser translates the response into an executable Python module. At runtime, each assertion is answered by the same model, prompted with the agent's current partial observation (visible objects, their states and relations) and asked to reply only `True` or `False`; a false assertion triggers its recovery actions. ProgPrompt is therefore closed-loop only through these pre-written recovery branches: the model is never asked to revise the plan. Each task costs one plan call plus one call per assertion reached during execution.

Our implementation follows the original prompt structure with two documented changes. The preamble asks for plain Python without Markdown fences, and a response consisting of exactly one fenced block is unwrapped before parsing (both the raw format compliance and the unwrapping are recorded). Generated code that imports an unsupported action, fails to parse or crashes is recorded as a failed task rather than aborting the run.

4.3 Baseline 2: Code as Policies

Code as Policies (CaP) has the model write a *policy*: ordinary Python whose control flow reads observations and reacts to action outcomes (Liang et al. 2023). Our adaptation keeps its structure of *language model programs* (LMPs), each a separate prompt that returns code:

- the *policy LMP* writes `policy()` for the task;

```
def policy():
  salmon = parse_objects("visible salmon")
  if not salmon:
    result = find("salmon")
    if result.status != "succeeded":
      return False
    salmon = parse_objects("visible salmon")
  if not salmon or not acquire_object(salmon[0]):
    return False
  result = find("fridge")
  if result.status != "succeeded":
    return False
  fridges = parse_objects("visible fridges")
  if not fridges or not place_held_in_container(
      salmon[0], fridges[0]):
    return False
  return close(fridges[0]).status == "succeeded"
```

Listing 2: The CaP version of the training program in Listing 1, abridged. `acquire_object` and `place_held_in_container` are helpers that the function-generating LMP writes during dependency resolution before the policy is executed.

- the *function-generating LMP* writes any helper that the policy calls but does not define, recursively and depth-first, given the call site and the calling program;
- two *specialised LMPs*, `parse_objects` and `parse_question`, are invoked by the policy at run time; each writes query code over the current observation that returns the matching visible objects or the answer to a question.

The policy prompt lists the 24 actions, six observation queries (visible objects, an object's states, the contents of a container or surface, whether an object is held, and the current room), the object classes and rooms of the scene, the agent's current observation, three example policies and the task (Listing 2). Actions return a result whose status (`succeeded`, `failed` or `skipped`) the policy must check explicitly. CaP receives the status and error category; AP additionally receives a detailed error message. References used in actions must identify objects that are currently observed or held.

Generated code is parsed and checked before it runs: imports, private names, reflection, redefining an API name and attributes outside a small allow-list are rejected, and only basic built-ins are available. The checked code runs in a separate process that has no access to the simulator, evaluator or ground truth; every action and query is a remote call that the parent process executes and logs. Missing helper dependencies are resolved recursively before each generated program is executed. The specialised query LMPs can still be invoked while the policy runs; their generated query programs undergo the same validation and dependency resolution.

We rewrote the 24 ProgPrompt training programs as CaP policies, so both methods are given the same three example tasks. The agent starts in the same room, with the same inherited exclusion of `grab(wallphone)` from execution, and scoring and task accounting are shared with ProgPrompt.

Both baselines use feedback, but in different forms. ProgPrompt follows pre-written assertion-recovery branches; CaP can use ordinary Python to query observations, branch, retry or execute fallback behaviour. Our CaP adaptation does not provide a runtime mechanism for model-generated policy repair: responses to failure must already be expressed in the generated code. Returning `False` is one possible outcome, rather than the only behaviour CaP can express. Each task costs one policy call, one call per generated helper and one call per specialised-LMP invocation.

4.4 Proposed method: Adaptive Policies

Adaptive Policies (AP) builds on CaP's LMP structure, observation API and isolated execution, but adds model-assisted repair. Its policy prompts and examples replace early-return guards with task code and state the expectations that later steps rely on as *checkpoints*, as in this fragment of the AP version of the training program in Listings 1 and 2:

```
acquire_object(salmon)
checkpoint("holding the salmon",
  lambda: is_holding(salmon))
find("fridge")
checkpoint("a fridge is visible", lambda: any(
  obj.name == "fridge"
  for obj in get_visible_objects()))
```

In `checkpoint(description, check)`, the description names the expected state and `check` is a zero-argument callable. The prompts encourage checks that use observation queries only. A checkpoint returns the check's truthy value, so it can also select objects. Evaluating such a check requires no model invocation; using a specialised LMP inside the check would incur its normal inference cost. Repair calls are triggered by a failed checkpoint, a failed action or a recoverable execution exception, depending on the checkpoint condition. Policy generation, helper generation and specialised query LMPs retain their own model calls.

AP's action feedback additionally includes the simulator precondition or rejection message, whereas CaP receives only status and category. AP also permits continuation after a known native rejection and has a larger model-call budget and task deadline (Experimental Setup). The comparison therefore evaluates the complete repair method, including these feedback and budget differences, rather than checkpoints alone. Listing 3 illustrates the policy, local recovery and re-planning stages from an evaluation episode.

Failure context. At a failure, the model receives the same API description and current observation as the policy LMP, plus four sections built by the runtime:

1. the running policy and helpers, with the failing line and its callers marked;
2. the failure: a checkpoint description and check source, a failed action and its error message, or an exception;

```python
def policy():
  toaster = parse_objects("toaster")[0]
  bread = parse_objects("breadslice")[0]
  find(toaster)
  grab(bread)   # fails: not close to the bread
  putback(bread, toaster)
  switchon(toaster)

# Local recovery for grab(bread), written twice;
# rejected: it reassigns the bound name bread.
def recover():
  bread = parse_objects("breadslice")[0]
  walk(bread)
  grab(bread)

# Re-plan: continues from the current state.
def policy():
  bread = parse_objects("breadslice")[0]
  toaster = parse_objects("toaster")[0]
  find(bread)
  grab(bread)
  putback(bread, toaster)
  switchon(toaster)
```

Listing 3: Code generated by Gemma 4 12B (implicit placement) for *make toast* in the env2 scene: the initial policy, its rejected local recovery and the continuation policy (the model's comments removed; ours added). Figure 3 reports the execution and outcome.

3. the execution history: actions and their outcomes, passed checkpoints and earlier recovery attempts; and
4. the caller's local variables holding object references or plain values, which local recovery code may use by name.

Recovery ladder. Each failure escalates only as far as needed:

1. *Local recovery.* The recovery LMP writes a `recover()` function, which runs from the current state. A failed checkpoint is then checked again; a failed action is retried, unless `recover()` returns `True` to report that the action's effect already holds. Each failure site gets at most two local recoveries.
2. *Helper regeneration.* The innermost generated helper on the call stack is rewritten with the same name and parameters, given the failure context, and called again (at most twice per helper).
3. *Re-planning.* The re-planning LMP writes a continuation `policy()` that completes the rest of the task from the current state; it replaces the running policy within the same episode.

An exception in `policy()` itself goes straight to re-planning. Code written during a recovery cannot place checkpoints, and failures inside it are not recovered again. If the same failure recurs after a re-plan without any action having succeeded in between, the episode ends as *making no progress*. Since AP is designed to continue after a failed action, a script that the simulator rejects does not end its episode, as it does for the baselines.

Repair applies to failures encountered while validated policy code executes. Initial syntax and validation errors do not enter this recovery mechanism. An invalid local recovery counts as an unsuccessful attempt and can cause escalation within the configured limits. A normal return from `policy()` ends execution; there is no mandatory final goal checkpoint. Goal achievement is determined separately by the shared evaluator.

Checkpoint placement. We compare three conditions to examine explicit checks and action-failure recovery within AP:

- **Explicit:** failed model-written checkpoints trigger repair; failed action results do not automatically trigger repair.
- **Implicit:** failed action results trigger repair; prompts and examples do not include explicit checkpoints.
- **Both:** failed explicit checkpoints and failed action results trigger repair.

Recoverable execution exceptions can trigger repair in all three conditions.

The policy examples are the 24 CaP training policies rewritten without fallback branches and with checkpoints (which are removed for the implicit placement), and the recovery and re-planning prompts each have their own examples. A task costs one policy call, the helper and object-parsing calls of CaP, and one call per local recovery, helper regeneration and re-planning.

5 Experimental Setup

Tasks and reference repairs. The training pool contains 24 tasks. Evaluation has 45 entries: 10 seen, 10 unseen and 5 ambiguous-goal tasks in scene 0, and 10 tasks in each of scenes 1 and 2. Names can recur across scenes; these are not 45 distinct goals. Source split labels do not imply that every "seen" target occurs among the selected prompt examples. Explicit mappings and repaired graph facts define a revised benchmark, not an exact reproduction of ProgPrompt.

Claude Code and Codex assisted with experiment design and repairs to inconsistent task graphs. Original references are archived; corrected annotations were validated by simulator replay. Repair provenance and the task-suite table are in Appendix B.

Models and conditions. Qwen3.5-9B and Gemma 4 12B run through LM Studio with Q4_K_M weights and a configured 64k-token context (Qwen Team 2026; Google 2026b). They use RTX 3060 and RTX 3090 GPUs, respectively. Within each model, weights remain fixed while requests set `reasoning_effort` to `none` or `medium`; these toggle reasoning rather than establish a shared calibrated effort budget. Temperatures $\{0, 0.6, 1.0\}$ crossed with these settings give six conditions.

Code requests allow 8192 tokens and 600 s, doubled for the ambiguous suite. ProgPrompt assertions instead use temperature 0, 128 tokens and reasoning disabled. Initial policy prompts share three training identities sampled with seed

Per-task limit	ProgPrompt	CaP	AP
Deadline (s)	—	1200	1800
Ambiguous deadline (s)	—	2400	3600
Model requests	—	32	64
Action requests	—	100	100
Observation queries	—	1000	1000
Generated helpers	—	24	24
Helper/LMP depth	—	8	8

Table 1: Additional runtime limits. A dash means no corresponding task-wide cap; per-request limits still apply. Action requests include failed attempts.

42; helper, query and repair prompts have their own examples. Table 1 gives task-wide limits. SDK requests and invalid initial programs are not automatically retried; AP recovery is deliberate and budgeted. Simulator-aborted campaigns can be restarted once, with the superseded artifacts archived. Each task-condition in the final campaigns has one attempt: 540 trials per baseline and 1620 across AP's three conditions, 2700 in total. Detailed execution controls are in Appendix B.

Metrics and cost. Let F_i^0, F_i^*, F_i be initial, reference-final and achieved class-name fact sets, and $D_i = F_i^* \setminus F_i^0$. For nonempty D_i,

$$\mathrm{PSR}_i = \frac{|D_i \cap F_i|}{|D_i|}, \quad \mathrm{SR}_i = \mathbf{1}[D_i \subseteq F_i]. \tag{3}$$

Exec is successful actions divided by attempted actions (0 for no attempts). SR and PSR are 0 for empty D_i. Headline scores average all intended trials; missing results contribute 0, while a trustworthy final state can retain its measured score after execution failure. Precision measures preservation of initially true reference facts and is reported only over measured states. Single-reference, class-name scoring can reject reasonable alternative goals.

Mean calls include all recorded purposes and intended trials. ProgPrompt action counts cover main-plan actions in normal-stop generations; CaP/AP counts issued action requests, including failures, over all trials. Timing and tokens cover the initial plan for ProgPrompt versus all model requests for CaP/AP; these scopes and different GPUs limit direct latency comparisons. Provider-reported reasoning tokens are not added again to completion totals. Full definitions and missing-value conventions are in Appendix B.

AP's first-failure score evaluates its state before recovery, or the final state if nothing fails. It is a within-episode diagnostic, not an independent no-recovery ablation. Outcome categories prioritise reference-goal success over runtime termination; resource-limit stops do not alone prove non-termination.

6 Results

Table 2 compares the methods across all six conditions. Figure 2 shows condition-level performance. These are descriptive comparisons: tasks recur across settings and AP placements, and there are no independent repeated attempts for each task-condition. Condition-level tables and extended diagnostics are in Appendix C; suite-level results are in Appendix F.

	Qwen3.5-9B			Gemma 4 12B		
Method	SR	PSR	Calls	SR	PSR	Calls
ProgPrompt	27.0	**50.3**	4.4	21.9	**49.6**	3.9
CaP	13.0	39.3	3.3	8.9	17.6	3.3
AP, explicit	21.9	39.7	10.0	**29.3**	43.7	7.4
AP, implicit	**27.4**	47.0	10.2	23.3	41.2	8.3
AP, both	23.7	42.9	12.6	21.1	35.6	8.6
AP, all	24.3	43.2	10.9	24.6	40.2	8.1
AP scored at its first failure						
explicit	13.3	29.1		9.3	20.0	
implicit	13.0	31.2		1.9	7.1	
both	14.1	30.3		7.0	16.7	

Table 2: The three methods pooled over all conditions (%; 270 trials per row, 810 for "AP, all"). Calls: mean model calls per task. The last block scores the recorded state before the first recovery, or the final state if no failure occurs. This is a within-episode diagnostic. Bold marks each model's best SR and PSR.

	Calls	Acts	Asserts	Time	Tok.	Reas.	Trunc.
ProgPrompt							
Qwen, off	4.8	10.0	5.8	5.2	256	0	0
Qwen, on	4.0	8.6	4.8	25.1	1265	960	17
Gemma, off	4.9	7.5	5.1	3.4	199	0	0
Gemma, on	2.9	5.5	2.9	127.4	6067	5894	29
Code as Policies							
Qwen, off	3.6	3.3	–	9.5	353	0	1
Qwen, on	2.9	2.9	–	34.2	1657	1099	25
Gemma, off	2.7	2.3	–	7.7	313	0	0
Gemma, on	3.9	1.3	–	152.4	8192	8189	38
Adaptive policies (all placements)							
Qwen, off	12.6	11.5	–	18.9	726	0	31
Qwen, on	9.3	10.1	–	155.1	7560	4444	115
Gemma, off	11.0	13.4	–	13.1	474	0	4
Gemma, on	5.2	5.0	–	207.7	9092	8876	209

Table 3: Cost by method, model and reasoning mode (135 trials per baseline row, 405 per AP row). Calls: mean over all trials. Acts: main-plan actions excluding recovery branches over normal-stop ProgPrompt generations; issued action requests, including failed attempts, over all CaP/AP trials. Asserts: mean per normal-stop ProgPrompt generation. Time (s), Tok. and Reas.: median request time, completion tokens and reported reasoning tokens for the initial plan (ProgPrompt, available measurements) or summed across the episode (CaP/AP, all trials). Trunc.: token-limit finishes in the initial plan (ProgPrompt) or any call (CaP/AP); denominators are 135 and 405, respectively.

Task performance (RQ1). ProgPrompt succeeds in 73 of 270 Qwen trials (27.0%) and 59 of 270 Gemma trials

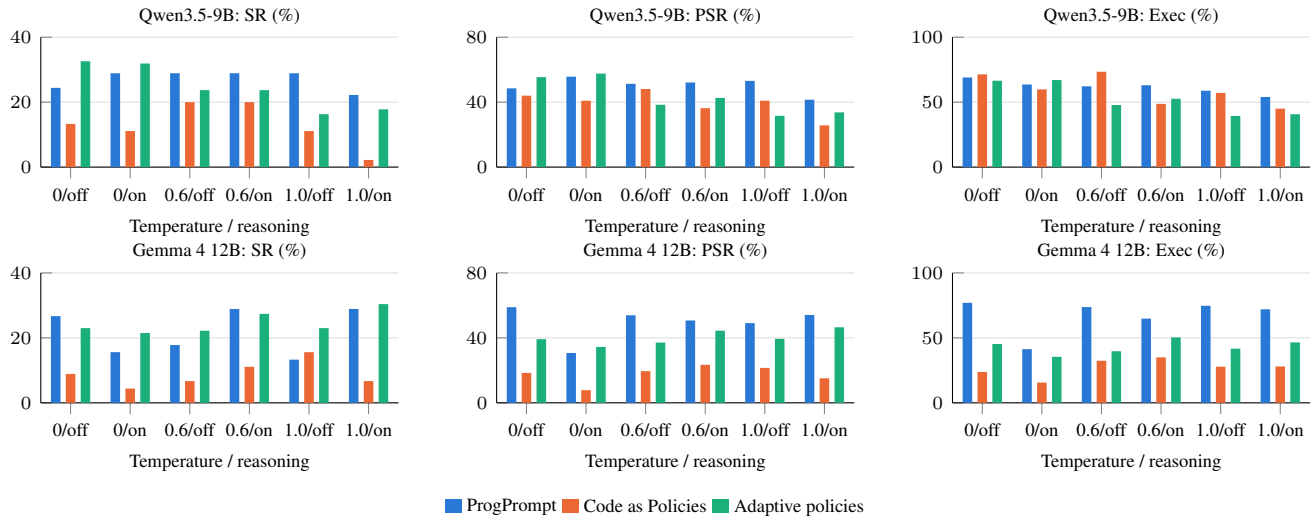


Figure 2: Success rate, partial success rate and executability per condition for the three methods, pooled over the 45 evaluation tasks (values in Tables 7, 8 and 9). Adaptive policies are pooled over the three checkpoint placements (135 trials per bar).

	ProgPrompt		CaP		AP	
Outcome	Qwen	Gem.	Qwen	Gem.	Qwen	Gem.
Success	73	59	35	24	197	199
Ran, goal not reached	151	174	155	158	159	170
policy gave up	–	–	129	131	–	–
Stopped, no progress	–	–	–	–	82	147
Truncated output	17	27	26	37	104	163
Invalid syntax	10	9	7	4	42	7
Invalid API use	19	1	45	12	190	82
Other run-time error	0	0	1	22	3	1
Resource limit	0	0	1	13	33	41
Total	270	270	270	270	810	810

Table 4: Mutually exclusive operational outcomes (AP pooled over placements). Reference-goal success takes priority over runtime termination; unsuccessful normal returns are then separated from error outcomes. "Policy gave up" is a subset of "Ran, goal not reached": CaP returned `False`. Error categories follow the analysis scripts' priority rules; a recorded truncation can coexist with another error. No progress means a repeated failure after re-planning without an intervening successful action. Invalid API use includes argument and policy-contract violations. Resource limit means a nesting, call, action, query, code-size or time limit was reached; it does not by itself establish non-termination.

(21.9%). CaP succeeds in 35 (13.0%) and 24 (8.9%), respectively. AP succeeds in 197 of 810 Qwen trials (24.3%) and 199 of 810 Gemma trials (24.6%). AP's pooled SR is therefore 2.7 percentage points lower than ProgPrompt for Qwen and 2.7 higher for Gemma, while ProgPrompt retains the highest PSR and Exec for both models. We make no equivalence or significance claim. Implicit placement has Qwen's highest observed AP SR (27.4%); explicit placement has Gemma's (29.3%).

AP's first-failure snapshot scores are 13.5% and 6.0% SR, compared with 24.3% and 24.6% at the end. Of its successes, 109 of 197 Qwen trials and 163 of 199 Gemma trials include a failure, and 61 and 80 include re-planning. These episodes show progress after repair, without isolating recovery from the methods' other differences. Figure 3 and Listing 3 provide an illustrative trace.

No method achieves the annotated goals in the ambiguous suite: 0 of 60 trials per baseline and 0 of 180 for AP. This does not imply that every generated interpretation is unreasonable. Suite differences also confound scene, task composition and annotation; the high scores on env1/env2 are not an isolated test of transfer.

Reasoning and inference cost (RQ1–RQ2). Reasoning changes ProgPrompt SR from 27.4% to 26.7% for Qwen and from 19.3% to 24.4% for Gemma, while PSR and Exec decrease. CaP's pooled SR decreases for both models. AP's changes from 24.2% to 24.4% for Qwen and 22.7% to 26.4% for Gemma, with variation across checkpoint conditions: Gemma's implicit placement changes from 13.3% to 33.3%, but its other placements decrease. No single setting is best across methods.

AP uses a mean of 10.9 Qwen and 8.1 Gemma calls per trial, with medians of 5 and 4. ProgPrompt's means are 4.4 and 3.9; CaP's are 3.3 for both. Long repair sequences increase AP's means. Median initial-plan request time for ProgPrompt changes from 5.2 to 25.1 s (Qwen) and 3.4 to 127.4 s (Gemma) with reasoning. CaP's episode-wide request-time medians increase 3.6- and 20-fold; AP's rise from 19 to 155 s and 13 to 208 s. These scopes differ. Table 3 reports the detailed costs.



(a) `find(toaster)` first policy; then `grab` fails
(b) `find(bread)` re-planned policy
(c) `grab(bread)` re-planned policy
(d) `putback(bread, toaster)` re-planned policy
(e) `switchon(toaster)` re-planned policy

Figure 3: An adaptive-policy episode on *make toast* in the env2 scene (Listing 3; Gemma 4 12B, implicit placement, temperature 0.6, reasoning off). Frames are the simulator's views after each successful action. The first policy finds the toaster (a) but tries to grab the bread from where it stands, and the grab fails because the agent is not close to it. Both local recoveries are rejected by the code checks, so the failure escalates: the model re-plans from the current state, given the program, the failure and the recovery history, and the continuation policy finds the bread (b), grabs it (c), puts it in the toaster (d) and switches the toaster on (e). The task succeeds with eight model calls. With the same model and settings, the CaP policy's `grab` failed in the same way, but the policy did not check it and returned `True` without making toast; the ProgPrompt plan ends by grabbing the bread again after switching the toaster on. Neither baseline completed *make toast* in any of its 36 trials across three scenes; AP completed it in 4 of 108.

Observed failures (RQ3). Most unsuccessful baseline trials return without the reference goal. Many CaP policies return `False` after a failed guard, while some index empty object-query results before acting. This describes the generated programs, not a restriction on CaP's ability to express fallback behaviour. Invalid or truncated initial code prevents runtime repair in about a quarter of AP trials; another 180 terminate with invalid or truncated continuations. With reasoning, at least one call reaches its output cap in 115 of 405 Qwen and 209 of 405 Gemma AP trials, versus 25 and 38 of 135 CaP trials. These trial-level counts are not per-call truncation probabilities.

AP has 243 no-progress terminations (91 Qwen, 152 Gemma), but 14 already satisfy the reference goal. The unsuccessful-outcome table therefore reports 229 (82 and 147). Another 74 unsuccessful AP trials stop at resource limits. Runtime completion, goal success and the assigned error category are distinct; Table 4 and Appendix C explain the classification and give the diagnostic breakdowns.

7 Discussion

The results support model-assisted repair as a way to recover from some execution failures. AP has higher pooled success than our CaP adaptation, and many successful episodes include repair. Its comparison with ProgPrompt is mixed, and the experiment evaluates a complete method: prompting, richer error feedback, continued execution after known rejections and larger budgets change together. First-failure snapshots describe subsequent progress but do not isolate a causal recovery effect.

Generated code can use observations without producing a successful fallback. Many CaP policies return early; ProgPrompt often executes without achieving the complete reference goal. AP supplies another opportunity to continue, at the cost of additional inference. A failure-free AP episode needs no repair call, but its total policy and query costs need not equal CaP's. No cost advantage over per-step replanning is established because that baseline was not evaluated.

The highest-scoring checkpoint condition depends on the model. Counts and recovery rates alone do not establish checkpoint quality because conditions generate different programs and encounter different failures. Reasoning also has mixed effects under the evaluated output budget. Long traces and truncations are consistent with competition between reasoning and code, but do not fully explain failure. Invalid initial and continuation programs remain outside the reach of the current execution-time repair mechanism.

Limitations. The main limits are:

- **Coverage:** one attempt per task-condition, small suites and repeated task identities limit uncertainty and generalisation claims.
- **Scoring:** one annotated outcome and class-name facts can reject reasonable alternatives and omit instance-level distinctions.
- **Setting:** a modified simulator, repaired references and symbolic observations do not test learned perception or external changes.
- **Comparisons:** CaP is our adaptation; AP also changes feedback, prompts and budgets. Models are 4-bit quantised and use different GPUs; some cost measures have different scopes.

Appendix D expands these limitations and interpretations.

8 Future Work

Repeated trials and controlled ablations could separate the effects of repair, feedback and budget choices. Matched perturbations would test responses to external changes rather

than only generated-policy errors. Bounded feedback on invalid or truncated code could extend repair to generation, while multiple reviewed goal interpretations could improve ambiguous-task evaluation. Further extensions could examine perception-derived observations and supervised physical execution, distinguishing observation accuracy, checkpoint correctness and recovery success. Appendix E sets out these proposed evaluations.

9 Conclusion

We evaluated Adaptive Policies, which extends generated Python code policies with checkpoints and escalating model-assisted repair, against ProgPrompt and our VirtualHome adaptation of Code as Policies. Across two open-weight models, Adaptive Policies achieved pooled task success rates of 24.3% and 24.6%, compared with 13.0% and 8.9% for CaP. Its comparison with ProgPrompt was mixed, and ProgPrompt retained higher partial success and executability. Adaptive Policies required more model calls: medians of five and four per task, with means of 10.9 and 8.1. Reasoning effects varied across methods and settings while increasing generation time, and no method achieved the annotated goals on the ambiguous suite.

These results support model-assisted repair as a way to recover from some execution failures, with additional inference cost. The comparison evaluates the complete method, including its prompting, feedback and budget choices. Invalid and truncated policies remain a limitation because execution-time repair depends on code that can run.

References

Besta, M.; Blach, N.; Kubicek, A.; Gerstenberger, R.; Podstawski, M.; Gianinazzi, L.; Gajda, J.; Lehmann, T.; Niewiadomski, H.; Nyczyk, P.; et al. 2024. Graph of thoughts: Solving elaborate problems with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, 17682–17690.

Brohan, A.; Chebotar, Y.; Finn, C.; Hausman, K.; Herzog, A.; Ho, D.; Ibarz, J.; Irpan, A.; Jang, E.; Julian, R.; et al. 2023. Do as i can, not as i say: Grounding language in robotic affordances. In *Conference on robot learning*, 287–318. PMLR.

Google. 2026a. Gemma 4 12B Instruction-Tuned (GGUF). https://huggingface.co/lmstudio-community/gemma-4-12B-it-GGUF.

Google. 2026b. Gemma 4 12B Instruction-Tuned Model Card. https://huggingface.co/google/gemma-4-12B-it.

Gundawar, A.; Verma, M.; Guan, L.; Valmeekam, K.; Bhambri, S.; and Kambhampati, S. 2024. Robust planning with llm-modulo framework: Case study in travel planning. *arXiv preprint arXiv:2405.20625*.

Guo, D.; Yang, D.; Zhang, H.; Song, J.; Zhang, R.; Xu, R.; Zhu, Q.; Ma, S.; Wang, P.; Bi, X.; et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*.

Hazra, R.; Dos Martires, P. Z.; and De Raedt, L. 2024. Saycanpay: Heuristic planning with large language models using learnable domain knowledge. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, 20123–20133.

Huang, J.; Chen, X.; Mishra, S.; Zheng, H. S.; Yu, A. W.; Song, X.; and Zhou, D. 2024. Large Language Models Cannot Self-Correct Reasoning Yet. In *The Twelfth International Conference on Learning Representations*.

Huang, W.; Abbeel, P.; Pathak, D.; and Mordatch, I. 2022a. Language models as zero-shot planners: Extracting actionable knowledge for embodied agents. In *International conference on machine learning*, 9118–9147. PMLR.

Huang, W.; Xia, F.; Xiao, T.; Chan, H.; Liang, J.; Florence, P.; Zeng, A.; Tompson, J.; Mordatch, I.; Chebotar, Y.; Sermanet, P.; Jackson, T.; Brown, N.; Luu, L.; Levine, S.; Hausman, K.; and brian ichter. 2022b. Inner Monologue: Embodied Reasoning through Planning with Language Models. In *6th Annual Conference on Robot Learning*.

Kaelbling, L. P.; Littman, M. L.; and Cassandra, A. R. 1998. Planning and Acting in Partially Observable Stochastic Domains. *Artificial Intelligence*, 101(1–2): 99–134.

Kambhampati, S.; Valmeekam, K.; Guan, L.; Verma, M.; Stechly, K.; Bhambri, S.; Saldyt, L.; and Murthy, A. 2024. Position: LLMs can't plan, but can help planning in LLM-modulo frameworks. In *Proceedings of the 41st International Conference on Machine Learning*, ICML'24. JMLR.org.

Katz, M.; Kokel, H.; Srinivas, K.; and Sohrabi, S. 2024. Thought of Search: Planning with Language Models Through The Lens of Efficiency. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.

Kolve, E.; Mottaghi, R.; Han, W.; VanderBilt, E.; Weihs, L.; Herrasti, A.; Deitke, M.; Ehsani, K.; Gordon, D.; Zhu, Y.; Kembhavi, A.; Gupta, A.; and Farhadi, A. 2017. AI2-THOR: An Interactive 3D Environment for Visual AI. *arXiv preprint arXiv:1712.05474*.

LaValle, S. M. 2006. *Planning Algorithms*. Cambridge University Press.

Liang, J.; Huang, W.; Xia, F.; Xu, P.; Hausman, K.; Ichter, B.; Florence, P.; and Zeng, A. 2023. Code as Policies: Language Model Programs for Embodied Control. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, 9493–9500.

Liu, B.; Jiang, Y.; Zhang, X.; Liu, Q.; Zhang, S.; Biswas, J.; and Stone, P. 2023. LLM+P: Empowering Large Language Models with Optimal Planning Proficiency. *arXiv preprint arXiv:2304.11477*.

Puig, X.; Ra, K.; Boben, M.; Li, J.; Wang, T.; Fidler, S.; and Torralba, A. 2018. VirtualHome: Simulating Household Activities via Programs. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 8494–8502.

Qwen Team. 2026. Qwen3.5-9B Model Card. https://huggingface.co/Qwen/Qwen3.5-9B. GGUF build: lmstudio-community/Qwen3.5-9B-GGUF.

Rana, K.; Haviland, J.; Garg, S.; Abou-Chakra, J.; Reid, I.; and Sünderhauf, N. 2023. SayPlan: Grounding Large Language Models using 3D Scene Graphs for Scalable Robot Task Planning. In *7th Annual Conference on Robot Learning*.

Savva, M.; Kadian, A.; Maksymets, O.; Zhao, Y.; Wijmans, E.; Jain, B.; Straub, J.; Liu, J.; Koltun, V.; Malik, J.; et al. 2019. Habitat: A platform for embodied ai research. In *Proceedings of the IEEE/CVF international conference on computer vision*, 9339–9347.

Shridhar, M.; Thomason, J.; Gordon, D.; Bisk, Y.; Han, W.; Mottaghi, R.; Zettlemoyer, L.; and Fox, D. 2020. Alfred: A benchmark for interpreting grounded instructions for everyday tasks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 10740–10749.

Singh, I.; Blukis, V.; Mousavian, A.; Goyal, A.; Xu, D.; Tremblay, J.; Fox, D.; Thomason, J.; and Garg, A. 2023. Progprompt: Generating situated robot task plans using large language models. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, 11523–11530. IEEE.

Song, C. H.; Wu, J.; Washington, C.; Sadler, B. M.; Chao, W.-L.; and Su, Y. 2023. LLM-Planner: Few-Shot Grounded Planning for Embodied Agents with Large Language Models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2998–3009.

Valmeekam, K.; Marquez, M.; Olmo, A.; Sreedharan, S.; and Kambhampati, S. 2023a. Planbench: An extensible benchmark for evaluating large language models on planning and reasoning about change. *Advances in Neural Information Processing Systems*, 36: 38975–38987.

Valmeekam, K.; Marquez, M.; Sreedharan, S.; and Kambhampati, S. 2023b. On the Planning Abilities of Large Language Models - A Critical Investigation. In Oh, A.; Naumann, T.; Globerson, A.; Saenko, K.; Hardt, M.; and Levine, S., eds., *Advances in Neural Information Processing Systems*, volume 36, 75993–76005. Curran Associates, Inc.

Valmeekam, K.; Stechly, K.; and Kambhampati, S. 2024. LLMs Still Can't Plan; Can LRMs? A Preliminary Evaluation of OpenAI's o1 on PlanBench. In *NeurIPS 2024 Workshop on Open-World Agents*.

Wang, X.; Wei, J.; Schuurmans, D.; Le, Q. V.; Chi, E. H.; Narang, S.; Chowdhery, A.; and Zhou, D. 2023. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *The Eleventh International Conference on Learning Representations*.

Wei, J.; Wang, X.; Schuurmans, D.; Bosma, M.; Ichter, B.; Xia, F.; Chi, E.; Le, Q.; and Zhou, D. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems*, volume 35.

Yao, S.; Yu, D.; Zhao, J.; Shafran, I.; Griffiths, T. L.; Cao, Y.; and Narasimhan, K. 2023a. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. In *Advances in Neural Information Processing Systems*, volume 36.

Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; and Cao, Y. 2023b. ReAct: Synergizing Reasoning and Acting in Language Models. In *The Eleventh International Conference on Learning Representations*.

A Extended background

LLM planning and grounding. LLMs can generate candidate plans from task descriptions and examples, but their outputs need not satisfy action preconditions or achieve the goal (Valmeekam et al. 2023a,b). Grounding approaches map generated steps to available actions (Huang et al. 2022a) or score candidate actions using learned affordances (Brohan et al. 2023; Hazra, Dos Martires, and De Raedt 2024). Another approach translates a language description into the Planning Domain Definition Language (PDDL) and delegates search to a classical planner, as in LLM+P (Liu et al. 2023). A sound planner's guarantees apply to the formal model it receives; successful execution additionally depends on the correctness of the translation and the model's correspondence to the environment. Hierarchical approaches such as SayPlan combine language-based task planning with lower-level planning and execution feedback (Rana et al. 2023).

Program-based plans and policies. ProgPrompt represents tasks as Python-like programs using available action imports, environment objects and example plans. Its assertions test expected conditions and enable recovery actions when those conditions fail (Singh et al. 2023). It therefore already incorporates execution feedback rather than simply producing an open-loop action sequence. Code as Policies generates Python policy code that composes perception and control APIs through conditions, loops and recursively generated helper functions (Liang et al. 2023). Such code can implement the observation-dependent behaviour represented by π, but generating a policy does not establish that its code will run or achieve the goal. Adaptive Policies builds on this representation by adding checkpoints and repair that escalates from local recovery to helper regeneration and task replanning.

Execution feedback and repair. ReAct interleaves reasoning, actions and observations (Yao et al. 2023b); Inner Monologue incorporates success detection and scene descriptions into subsequent planning (Huang et al. 2022b); and LLM-Planner uses environmental feedback for grounded re-planning (Song et al. 2023). External verification offers another source of feedback: LLM-Modulo combines candidate generation with checks by external critics or verifiers (Kambhampati et al. 2024). This differs from asking a model to revise its answer without new evidence, which has proved unreliable on the reasoning tasks studied by Huang et al. (2024). These approaches motivate our focus on when to invoke model-assisted repair. A condition checked in executing code need not require inference, whereas asking a model to interpret feedback or revise a policy does.

Reasoning and inference cost. Chain-of-thought prompting elicits intermediate reasoning within a generated response (Wei et al. 2022); it need not require additional model calls. Self-consistency samples multiple reasoning paths (Wang et al. 2023), while Tree of Thoughts and Graph of Thoughts organise generation and evaluation over alternative intermediate solutions (Yao et al. 2023a; Besta et al. 2024). These methods report gains on their respective reasoning tasks, rather than establishing a general improvement in embodied planning. Their costs also differ: longer responses consume more tokens, while sampling and search can require additional invocations. Model calls, token use and generation time therefore measure different aspects of inference cost (Katz et al. 2024). Reasoning language models such as DeepSeek-R1 are trained to produce extended reasoning before answering (Guo et al. 2025). On PlanBench, o1 improved substantially over the evaluated LLMs, but performance remained sensitive to problem size and changes to domain terminology (Valmeekam, Stechly, and Kambhampati 2024). The two models used here, Qwen3.5-9B and Gemma 4 12B, support enabling and disabling extended reasoning (Qwen Team 2026; Google 2026b). We compare these settings within each model while keeping its weights fixed.

Evaluation context. VirtualHome simulates household activities expressed as programs (Puig et al. 2018). Other resources include the Habitat simulation platform (Savva et al. 2019) and ALFRED, an instruction-following benchmark built on the AI2-THOR environment (Shridhar et al. 2020; Kolve et al. 2017). We use VirtualHome and the ProgPrompt task suites to evaluate the three methods under a shared protocol. ProgPrompt reports task success, goal-condition recall and executability (Singh et al. 2023); our metric definitions and failure accounting are detailed in the evaluation section. Consistent scoring supports comparisons within this study. Different models, repaired task references and simulator changes qualify comparisons with the original ProgPrompt results; this study is not an exact reproduction of those results.

B Detailed evaluation protocol

Task suites. We use VirtualHome and the repaired ProgPrompt datasets to compare the three methods under a shared protocol (Table 5). The 24 training tasks are only a pool of in-context examples. The five evaluation suites total 45 task entries: the original ProgPrompt seen and unseen test splits in scene 0, five unseen tasks with deliberately underspecified goals, and ten tasks in each of two further scenes. Task names in env1/env2 may overlap with the scene-0 suites, so the 45 entries are not 45 unique goals.

We retain the source dataset's seen/unseen split labels; "seen" does not mean that every target occurs among the three examples selected for a prompt. The comparison uses a revised benchmark version rather than an exact reproduction of the original ProgPrompt experiments.

Reference repairs and tool assistance. Repairs establish explicit task-to-graph mappings and correct inconsistent reference facts, including unsupported placement relations, inaccessible object instances and stale room or proximity relations. The original records are archived separately. The repository's mapping audit and validation records document the changes and reference replays: `docs/ground_truth_graph_mapping_audit.md` and `docs/dataset_validation.md`.

Suite	Scene	Tasks	Purpose
train	0	24	In-context example pool
test_seen	0	10	Seen tasks
test_unseen	0	10	Unseen tasks
ambiguous	0	5	Underspecified goals
env1	1	10	New environment
env2	2	10	New environment

Table 5: VirtualHome task suites. "Ambiguous" is `test_unseen_ambiguous_goals`.

Model	Params	GGUF size	GPU
Qwen3.5-9B	9B	5.6 GB	RTX 3060
Gemma 4 12B	12B	7.4 GB	RTX 3090

Table 6: Models (Qwen Team 2026; Google 2026a), quantised weight-file sizes and experiment GPUs. Generation-time comparisons across models also reflect the different hardware.

Claude Code and Codex were used to assist with experiment design and to identify and repair inconsistencies in the task graphs. The corrected references were checked by replaying their annotated programs in the simulator, as described in Methods. This validates the revised references under the chosen simulator and evaluator; comparisons with earlier results must account for these dataset changes.

Models. We use two hybrid open-weight models with the same weights, quantisation and tokenizer for their reasoning-enabled and reasoning-disabled conditions. Both run through LM Studio with 4-bit (Q4_K_M) GGUF weights and a configured 64k-token context window (Table 6). A third model, Qwen3.8-27B, was started but cancelled after 7 of 30 runs because of its generation speed; its partial results are in Appendix C.4.

Requests use `reasoning_effort: none` for reasoning disabled and `medium` for enabled. These are serving-interface settings, not a calibrated reasoning budget shared across models. The configured Qwen endpoint was probed to verify the off/on toggle and output cap; this did not establish distinct low/medium/high effort levels. The models' own reasoning controls are described in their model cards (Qwen Team 2026; Google 2026b).

Conditions. The design separates sampling settings, request limits and episode limits:

1. **Sampling and reasoning.** Each method and model uses six conditions: temperatures $\{0, 0.6, 1.0\}$ crossed with reasoning off/on. CaP's four LMP purposes and AP's six use the condition's settings. ProgPrompt assertions instead use temperature 0, 128 output tokens and reasoning disabled throughout.
2. **Request limits.** Every code-generating request allows 8192 output tokens and 600 s. The ambiguous suite uses 16384 tokens and 1200 s. These caps also apply to AP repair requests.
3. **Episode limits.** CaP and AP have additional deadlines and runtime caps (Table 1). AP has a larger request allowance and deadline to accommodate repair. ProgPrompt has no additional task-wide model-call or action-count cap.
4. **Example selection.** Initial plan/policy prompts use three examples sampled from the 24-task training pool with seed 42. Sampling is fixed rather than repeated per target, and the campaigns do not apply target-specific example exclusions. The same task identities are used across methods, with method-specific program representations. Helper, query and repair prompts have their own example collections.
5. **Execution controls.** `PYTHONHASHSEED` is 42 and all methods use the same checksum-verified simulator build. Runs execute as Slurm jobs on a shared GPU cluster. Seeds control example selection and Python hashing; they do not guarantee deterministic model or simulator execution.

There are no automatic SDK request retries or retries to repair an invalid initial program. AP's deliberate recovery and action retries are part of the method and consume its budgets. Separately, the launcher permits one restart of an aborted run after a simulator failure; the aborted artifacts are archived and superseded by the final run, not counted as additional experimental repeats.

Each task has one attempt per condition in the final campaigns. Thus each baseline has $2 \times 6 \times 45 = 540$ trials, and AP has $3 \times 540 = 1620$ over its three checkpoint conditions: 2700 trials in total. These are descriptive comparisons over the evaluated tasks and settings; there are no independent repeated attempts per task-condition pair from which to estimate repeat-to-repeat uncertainty.

Metrics. Following ProgPrompt (Singh et al. 2023), let F_i^0, F_i^* and F_i be the sets of class-name relation and object-state facts in task i's initial, reference-final and achieved final graphs. Define the target additions $D_i = F_i^* \setminus F_i^0$ and preserved reference facts $B_i = F_i^* \cap F_i^0$. For nonempty D_i,

$$\mathrm{PSR}_i = \frac{|D_i \cap F_i|}{|D_i|}, \qquad \mathrm{SR}_i = \mathbf{1}[D_i \subseteq F_i]. \tag{4}$$

The implementation assigns both scores 0 if D_i is empty. Executability is $\mathrm{Exec}_i = e_i/a_i$, where e_i counts successfully executed actions and a_i counts attempted actions under the shared task accounting; it is 0 if no action is attempted. The inherited metric named *precision* is $\mathrm{Prec}_i = |B_i \cap F_i|/|B_i|$ (0 for empty B_i). It measures preservation of initially true reference facts, not classification precision. Scoring uses class-name facts rather than instance IDs.

Headline SR, PSR and Exec are means over all intended task trials. Missing scores contribute 0; an execution failure with a trustworthy captured final state can retain its measured score. Precision is averaged only over trials with measured states, and its denominator is reported separately. The evaluator checks one annotated outcome: for ambiguous instructions, another reasonable interpretation may receive a

low score. These scores therefore measure agreement with the reference rather than all semantically valid outcomes.

Cost and output diagnostics. We report the following quantities with their measurement scopes:

- **Model calls:** all recorded planning and assertion calls for ProgPrompt, and all policy, helper, query and repair calls for CaP/AP; means include all intended trials.
- **Action counts:** ProgPrompt counts actions written in the main plan, excluding assertion-recovery branches, and averages over generations with a normal stop finish reason. CaP/AP counts issued action-API requests, including failed attempts and recovery actions, averaged over all trials. These are different measures, not directly comparable executed plan lengths.
- **Generation time:** ProgPrompt reports the initial plan request's elapsed time; CaP/AP sums elapsed model-request time across the episode. Medians use available plan timings for ProgPrompt and all trials for CaP/AP, with absent recorded time contributing 0. These are request timings, not complete task wall-clock or simulator execution times.
- **Token use:** ProgPrompt reports usage for the initial plan call; CaP/AP sums usage over all recorded calls. Completion-token totals and the provider's reasoning-token breakdown are reported separately, without adding the reasoning count again. Missing reasoning breakdowns are treated as 0 by the analysis, so they do not independently establish an absence of reasoning. Token medians follow the same available-plan versus all-trial distinction as timing.

We also record format compliance and token-limit finish reasons. For ProgPrompt, truncation refers to the initial plan; for CaP/AP, a trial is marked truncated if any recorded model call ends at the token limit.

Progress after the first failure. For AP, we additionally score its recorded state at the first failure before recovery, or its final state if no failure occurs. Comparing that score with the final score describes progress after recovery within the same episode. It is a diagnostic snapshot, not an independently executed no-recovery ablation or a separate CaP run.

C Detailed results and diagnostics

Table 2 summarises task performance (RQ1) and model calls (RQ2) across the three methods; Figure 2 shows per-condition performance. AP has higher pooled success than our CaP baseline, while its comparison with ProgPrompt varies by model and checkpoint condition. It also uses more model calls. The subsequent subsections examine these observations and the operational failure categories (RQ3).

Each comparison uses the protocol in Experimental Setup. The same tasks recur across conditions, and AP is additionally pooled across three checkpoint conditions. We report descriptive differences without independent-repeat uncertainty estimates or statistical significance claims.

		Qwen3.5-9B			Gemma 4 12B		
Temp.	Reas.	SR	PSR	Exec	SR	PSR	Exec
0	off	24.4	48.5	**69.0**	26.7	**58.9**	**77.0**
0	on	28.9	**55.7**	63.6	15.6	30.7	41.3
0.6	off	28.9	51.3	62.2	17.8	53.9	73.7
0.6	on	28.9	52.1	63.0	**28.9**	50.7	64.8
1.0	off	28.9	53.1	58.8	13.3	49.1	74.7
1.0	on	22.2	41.5	54.0	**28.9**	54.1	72.0
Reasoning off		27.4	50.9	63.3	19.3	54.0	75.2
Reasoning on		26.7	49.8	60.2	24.4	45.2	59.4
All		27.0	50.3	61.8	21.9	49.6	67.3

Table 7: ProgPrompt results (%) pooled over the 45 evaluation tasks per condition (135 per reasoning row, 270 per model). Every intended task is in the denominator. Bold marks each model's best value per metric; for Qwen, four conditions tie at the best SR (28.9%).

C.1 ProgPrompt

Table 7 and Figure 2 pool each condition over the 45 evaluation tasks; per-suite results are in Appendix F. Each task-condition has one attempt and the same tasks recur across settings. The results are descriptive comparisons, without claims of statistical significance or equivalence.

Task success and partial success differ. Across all conditions, Qwen3.5-9B completes 73 of 270 task trials (27.0%) and Gemma 4 12B 59 of 270 (21.9%). Mean per-task PSR is 50.3% and 49.6%, respectively, and mean per-task Exec is 62–67%. Precision over trials with measured final states is 99.8% for both models. This metric measures preservation of initially true reference facts rather than goal achievement.

No ambiguous-suite trial achieves the annotated goal. SR on the ambiguous suite is 0% in all 12 model–condition pairs (0 of 60 trials), despite the doubled output budget. PSR is 10% (Qwen) and 15% (Gemma). These tasks admit several reasonable end states but are scored against one annotated final graph. A reasonable alternative can therefore score as unsuccessful; the results do not establish that every generated interpretation is unreasonable. Among normal-stop generations, these plans have the highest mean main-plan action counts (17.7 and 11.6, against 5–10 on the other suites).

Reasoning effects differ across performance metrics. Averaged over temperatures, enabling reasoning changes SR from 27.4% to 26.7% for Qwen and from 19.3% to 24.4% for Gemma, while PSR and Exec fall in both models. Median initial-plan request time rises from 5.2 s to 25.1 s for Qwen and from 3.4 s to 127.4 s for Gemma (Table 3). Gemma's median plan response grows from 199 to 6067 completion tokens, with a median reported reasoning count of 5894. With reasoning enabled, 17 of 135 Qwen plans and 29 of 135 Gemma plans hit their configured output cap, against none with reasoning disabled. Gemma at temperature 0 with reasoning enabled truncates 18 of 45 plans and has the lowest PSR (30.7%) and Exec (41.3%) among its conditions.

ProgPrompt uses a small number of model calls. A ProgPrompt trial uses 2.9–4.9 model calls on average: an initial plan call and one call per assertion reached (Table 3). Reasoning-enabled conditions have fewer recorded assertion calls, while their normal-stop generations contain fewer assertions and main-plan actions. Early failures and truncation can also reduce the number of assertions reached. Assertion requests allow 128 tokens with reasoning disabled and total about 1 s per task. Their time is not included in the table's initial-plan timing column.

Most unsuccessful ProgPrompt trials miss the reference goal. Table 4 assigns each trial an operational outcome category. Plans that execute but do not reach the reference goal account for 151 of Qwen's 197 unsuccessful trials (77%) and 174 of Gemma's 211 (82%). The truncation category contains 17 and 27 trials, all with reasoning enabled; these mutually exclusive outcome counts differ from the count of all token-limit finishes. Prompts of about 2k tokens did not approach the 64k context window; the recorded length-limit finishes concern output budgets. Of Qwen's 19 invalid action uses, 16 call `find` with no object or two arguments, one calls `putin` without a destination, and two import unsupported actions (`eat`, `take`). Syntax or plan-structure rejections account for 10 Qwen and 9 Gemma trials. No ProgPrompt trial is classified as stopped for lack of progress.

Temperature effects vary with model and reasoning setting. For Qwen, SR ranges from 22% to 29%. For Gemma with reasoning disabled, SR decreases across temperatures (26.7%, 17.8%, 13.3%) while Exec remains at 74–77%. With reasoning enabled, the direction reverses and truncation counts decrease (18, 7 and 4). This association does not isolate the cause of the success-rate change.

Performance varies across task suites. Pooled over conditions, SR is highest on env1/env2 (33–43%) and, excluding the ambiguous suite, lowest on test_seen (Qwen 18.3%, Gemma 6.7%). Different tasks and reference annotations appear in each suite, so these results do not isolate transfer to new scenes from differences in task composition or annotation.

Output format. Qwen3.5-9B wraps 4 of 270 ProgPrompt plans in Markdown fences, all at temperature 1.0 with reasoning enabled; Gemma wraps none. Under CaP's longer prompt, Qwen fences at least one output in 87 of 135 reasoning-enabled trials. A response consisting of one fenced block is unwrapped under the shared protocol. Format compliance therefore varies with the model and prompt; these counts describe raw formatting rather than automatic task failure. Partial Qwen3.8-27B results and earlier parser diagnostics are reported in Appendix C.4.

C.2 Code as Policies

Table 8 gives the CaP results in the same layout as Table 7, and Figure 2 compares CaP with ProgPrompt condition by condition. Per-suite results are in Appendix F.

		Qwen3.5-9B			Gemma 4 12B		
Temp.	Reas.	SR	PSR	Exec	SR	PSR	Exec
0	off	13.3	44.0	71.4	8.9	18.4	23.8
0	on	11.1	40.9	59.8	4.4	7.8	15.6
0.6	off	**20.0**	**48.1**	**73.4**	6.7	19.5	32.4
0.6	on	**20.0**	36.3	48.7	11.1	**23.4**	**35.0**
1.0	off	11.1	40.9	57.1	**15.6**	21.5	27.8
1.0	on	2.2	25.7	45.0	6.7	15.1	28.0
Reasoning off		14.8	44.3	67.3	10.4	19.8	28.0
Reasoning on		11.1	34.3	51.2	7.4	15.4	26.2
All		13.0	39.3	59.2	8.9	17.6	27.1

Table 8: Code as Policies results (%), pooled as in Table 7. Bold marks each model's best value per metric.

CaP has lower pooled task success than ProgPrompt. CaP completes 35 of 270 trials with Qwen3.5-9B (13.0%, compared with 27.0% for ProgPrompt) and 24 of 270 with Gemma 4 12B (8.9%, compared with 21.9%). PSR and Exec are also lower: 39.3% and 59.2% for Qwen, and 17.6% and 27.1% for Gemma. CaP has higher Exec than ProgPrompt in 2 of 12 model–condition pairs (Qwen at temperatures 0 and 0.6 with reasoning disabled), and higher SR in one (Gemma at temperature 1.0 with reasoning disabled, 15.6% versus 13.3%). Precision over measured states is 99.9% for both models. No ambiguous-suite trial achieves its annotated goal (0 of 60).

Many generated CaP policies return before achieving the goal. Normal returns without reference-goal success account for 155 of Qwen's 235 unsuccessful trials and 158 of Gemma's 246. Of these, 129 and 131 return `False`, following guards for failed actions or unavailable objects as illustrated in Listing 2. These generated policies return instead of executing a successful fallback; the runtime supplies no model-generated policy repair. This does not mean that CaP cannot express fallback behaviour in its initial code.

In 54 Qwen and 90 Gemma trials, no action request is issued and Exec is 0, contributing to the low mean Exec. The agent initially observes the kitchen. Gemma sometimes indexes an empty object-query result without checking it, as in `parse_objects("window")[0]`, before moving (22 trials).

Recursive calls and resource-limit terminations. In 12 Gemma trials, all with reasoning enabled, the object-parsing LMP generates `ret_val = parse_objects("tv")`, recursively invoking itself until the nesting limit stops execution after eight nested calls. One Qwen trial reaches the 32-call limit and one Gemma trial reaches the task deadline. The action limit is not reached (at most 29 action requests). These observations identify recursion or resource exhaustion; a limit termination alone does not establish an infinite loop.

API and policy-contract errors occur in generated CaP code. The invalid-API category contains 45 Qwen trials with CaP, compared with 19 for ProgPrompt. Common cases

include testing an action result as a Boolean instead of checking its status (9), a missing or malformed `policy()` entry point (7), invalid argument types or references (10), and violations of code checks such as imports, forbidden attributes or redefining `find` (11). Eight of Gemma's 12 cases lack the required entry point. Syntax errors account for 7 Qwen and 4 Gemma trials. Actions accept class-name strings as well as valid object references; observation queries requiring an object expect a reference. The argument errors do not imply a general prohibition on class-name action arguments.

CaP uses a similar range of model-call counts. CaP uses 2.7–3.9 model calls per task on average, compared with ProgPrompt's 2.9–4.9 (Table 3). Object parsing accounts for 1.8 calls per task for Qwen and 2.2 for Gemma; `parse_question` is used in only two trials. Qwen generates helpers in 100 of 270 trials (0.5 helpers per task), while Gemma does so in 4 trials. CaP requests include the API description and permitted observation context. Total prompt-token usage has a median of about 5k for Gemma and 7k for Qwen.

Reasoning-enabled CaP has lower pooled success and higher cost. Enabling reasoning changes SR from 14.8% to 11.1% for Qwen and from 10.4% to 7.4% for Gemma, with lower PSR and Exec for both. Median episode-wide model-request time is 3.6 (Qwen) to 20 (Gemma) times higher. At least one token-limit finish occurs in 25 and 38 of 135 reasoning-enabled trials, compared with 1 and 0 of 135 reasoning-disabled trials. Qwen at temperature 1.0 with reasoning enabled has the lowest observed CaP SR, completing 1 of 45 trials. These are comparisons under the configured budgets.

Temperature and suites. The highest observed CaP SR occurs at temperature 0.6 for Qwen (20.0% with reasoning disabled or enabled) and temperature 1.0 with reasoning disabled for Gemma (15.6%). The new-environment suites have the highest observed SR for Qwen (env1 23.3%, env2 18.3%). Gemma's SR is 15.0% or lower on every suite. These suite comparisons do not isolate scene effects.

C.3 Adaptive Policies

Table 9 breaks AP down by condition and checkpoint placement. Figure 2 compares the methods by condition, with AP averaged over placements. Per-suite results are in Appendix F. The same tasks recur across placements and reasoning settings; the larger pooled trial count does not make these independent task samples.

AP has higher pooled success than our CaP baseline. AP completes 197 of 810 trials with Qwen3.5-9B (24.3%, compared with 13.0% for CaP) and 199 of 810 with Gemma 4 12B (24.6%, compared with 8.9%). PSR is higher for both models. Gemma's Exec is higher (43.2% versus 27.1%), while Qwen's is lower (52.3% versus 59.2%). These comparisons concern the full methods, including differences in prompts, error feedback, native-rejection handling and runtime budgets.

At the first-failure snapshot, AP's SR is 13.5% for Qwen and 6.0% for Gemma, compared with final SR of 24.3% and 24.6%. This describes progress within the recorded episodes rather than an independent no-recovery ablation. Of AP's successful trials, 109 of 197 (Qwen) and 163 of 199 (Gemma) encounter at least one failure; 61 and 80 of these include re-planning. Figure 3 illustrates such an episode. Precision over measured final states is 99.9%; this broad preservation metric does not isolate the effects of recovery on individual goal conditions.

The highest observed checkpoint condition differs by model. AP pooled over placements has SR 2.7 percentage points lower than ProgPrompt for Qwen and 2.7 points higher for Gemma. Implicit placement has the highest observed AP SR for Qwen (27.4%, versus ProgPrompt's 27.0%); explicit placement has the highest for Gemma (29.3%, versus 21.9%). These are descriptive differences, not tests of equivalence or significance. ProgPrompt has the highest PSR (50.3% and 49.6%) and Exec for both models. Programs rejected before acting receive Exec 0, so headline Exec reflects code-generation failures as well as action-level execution.

Qwen's explicit condition has 68 successful local recoveries out of 726 attempts (9%), compared with 230 of 992 (23%) in its implicit condition. These attempts encounter different failures and are not matched recovery tests. Gemma reaches an average of 3.3 explicit checkpoints per trial and has its lowest first-failure SR under implicit placement (1.9%). Neither checkpoint condition is best across both models.

Recovery adds model calls. AP uses 10.9 (Qwen) and 8.1 (Gemma) model calls per trial on average, compared with ProgPrompt's 4.4 and 3.9 and CaP's 3.3 for each model. Medians are 5 and 4 for AP, 3–4 for ProgPrompt and 2–3 for CaP. Long episodes increase the means; 35 AP trials reach the 64-call limit. Qwen averages 3.7 local-recovery calls, 1.9 re-plans, 0.7 helper-generation calls and 3.5 object-parsing calls per trial. Gemma averages 2.4, 1.0, 0.4 and 3.3, respectively. Local recovery is Qwen's largest call category; object parsing is Gemma's.

Repair prompts include the running program and failure context. Median total prompt-token usage is 12k (Qwen) and 10k (Gemma), about 1.7 and 2 times CaP's. A failure-free trial needs no repair call, but its initial policy and query costs can still differ from CaP because the prompts and generated behaviour differ.

Invalid generated code limits execution-time repair. In 220 Qwen and 194 Gemma trials (27% and 24%), no failure reaches the recovery ladder, usually because the initial policy is truncated, invalid or violates the policy contract. Another 121 Qwen and 59 Gemma trials end with a truncated or invalid continuation policy, which is not automatically regenerated again. Missing `policy()` entry points (65 and 52 trials) and code-check violations are common invalid uses.

		Qwen3.5-9B			Gemma 4 12B		
Temp.	Reas.	Explicit	Implicit	Both	Explicit	Implicit	Both
0	off	31/55/68	36/57/67	31/54/65	31/47/55	13/31/37	24/40/44
0	on	33/59/70	31/54/64	31/61/67	18/28/31	29/46/42	18/30/34
0.6	off	18/30/41	29/52/60	24/34/42	29/42/48	18/33/28	20/37/43
0.6	on	20/36/49	27/45/50	24/46/59	27/45/54	36/55/59	20/34/38
1.0	off	16/30/36	22/37/43	11/28/39	36/50/56	9/29/28	24/40/41
1.0	on	13/29/43	20/38/43	20/35/37	36/52/50	36/54/52	20/34/38
Reasoning off		21.5/38.3/48.3	28.9/48.6/56.4	22.2/38.6/49.0	31.9/46.1/53.3	13.3/30.9/30.6	23.0/38.7/42.8
Reasoning on		22.2/41.1/54.0	25.9/45.5/52.0	25.2/47.2/54.2	26.7/41.3/44.9	33.3/51.5/50.8	19.3/32.5/36.6
All		21.9/39.7/51.2	27.4/47.0/54.2	23.7/42.9/51.6	29.3/43.7/49.1	23.3/41.2/40.7	21.1/35.6/39.7

Table 9: Adaptive policies: SR / PSR / Exec (%) per condition and checkpoint placement, pooled over the 45 evaluation tasks (135 trials per reasoning row, 270 per placement).

At least one token-limit finish occurs in 115 of 405 Qwen and 209 of 405 Gemma reasoning-enabled AP trials, compared with 25 and 38 of 135 for CaP. An episode with more calls has more opportunities for a token-limit finish; these trial-level counts do not estimate per-call truncation probability. A truncated local recovery can count as a failed attempt followed by escalation rather than determining the final trial outcome.

No-progress termination is distinct from reference-goal failure. There are 243 no-progress terminations: 91 Qwen and 152 Gemma trials. The rule detects a repeated failure after re-planning with no intervening successful action. Nine Qwen and five Gemma trials already satisfy the annotated goal when stopped; they appear in the success row of Table 4. The unsuccessful no-progress counts are therefore 82 and 147, totalling 229. Runtime termination and reference-goal success measure different properties.

Repeated empty object-query selections occur in these episodes, but the aggregate termination label alone does not identify the underlying exception. Separately, 74 unsuccessful trials stop at resource limits: 33 at the call limit, 20 at the action limit, 15 at the deadline and 6 at query or code-size limits. These counts exclude resource-limit stops that already satisfy the reference goal. We do not infer how a no-progress episode would have evolved if that stopping rule had been disabled.

Reasoning effects vary by model and checkpoint condition. Enabling reasoning changes Qwen's pooled SR from 24.2% to 24.4% and Gemma's from 22.7% to 26.4%. For Gemma with implicit placement, SR changes from 13.3% to 33.3%, while its explicit and both conditions have lower SR with reasoning enabled (Table 9). These effects are descriptive and depend on the checkpoint condition. Median episode-wide model-request time rises from 19 to 155 s for Qwen and from 13 to 208 s for Gemma.

No ambiguous-suite AP trial achieves the annotated goal (0 of 180; Appendix G). The new-environment suites have the highest observed SR (33–44%), with task and annotation differences remaining confounded with scene. Qwen has its highest observed SR at temperature 0 (32.6% and 31.9% with reasoning disabled and enabled). No single temperature or reasoning setting is best across the evaluated methods.

C.4 Partial Qwen3.8-27B results

The 27B model (Q4_K_M, 19.4 GB, RTX 3090, ∼16 tokens/s) needed about 8.5 minutes for a plan that used the full 8192 tokens. The campaign was cancelled after 7 of 30 runs and replaced by Gemma 4 12B. The only condition completed on all five suites used temperature 0 with reasoning off. In this condition, the model reached SR 28.9%, PSR 57.9% and Exec 71.0% over 45 tasks, comparable to the two main models; every plan was returned in a code fence and unwrapped. On test_seen, temperature 0 with reasoning on gave SR 30% (2 plans truncated), and temperature 0.6 with reasoning off gave SR 30%. These runs are not used in the main comparison.

An earlier diagnostic with a strict parser rejected fenced responses and scored 0% because none parsed. This used a different parser protocol and is not included in the main comparison. The completed ambiguous-suite condition also had no annotated-goal successes.

D Extended interpretation and limitations

The baseline results distinguish execution from goal achievement. ProgPrompt has mean per-task PSR near 50%, but lower complete-task success. Its fixed recovery branches can address only the situations anticipated during generation. Many unsuccessful CaP policies instead return `False` after an unavailable object or failed action. Although CaP can express retries and fallback behaviour, these generated policies often stop without a successful fallback. Early returns may contribute to the performance gap, but the experiments do not isolate their effect from other differences in program structure and prompting.

For RQ1, AP has higher pooled success than our CaP baseline for both models. Its comparison with ProgPrompt is mixed: pooled SR is lower for Qwen and higher for Gemma, while ProgPrompt retains higher PSR and Exec. Gemma's explicit-checkpoint condition has higher observed SR than ProgPrompt, without establishing statistical superiority. Many successful AP episodes include repair, and

the first-failure snapshots show lower success than the final states. These observations support the usefulness of repair in the recorded episodes; they do not isolate its causal effect or establish equivalence between the initial AP policies and CaP. Figure 3 is an illustrative case: a continuation policy supplies a missing navigation step after local recovery attempts are rejected.

Checkpoint results also depend on the model. Implicit placement has Qwen's highest observed SR, whereas explicit placement has Gemma's highest. Checkpoint counts and recovery rates alone do not explain this difference: the conditions encounter different failures and generate different programs. Assessing checkpoint quality would require examining where checks are placed, what they test and which failures they expose, preferably with matched policy-level comparisons.

For RQ2, AP's improved success relative to CaP comes with more model calls and higher total prompt-token usage. Its call-count means exceed its medians, reflecting long repair sequences in some episodes. A failure-free episode requires no repair call, but its policy-generation and query costs can still differ from CaP's because the prompts and generated behaviour differ. The measured tradeoff is therefore higher success at additional inference cost relative to our CaP implementation. The study does not establish a cost advantage over per-step re-planning, which was not evaluated.

For RQ3, invalid initial and continuation policies limit what execution-time repair can accomplish. About a quarter of AP trials never reach the recovery ladder, usually because the initial policy is truncated or invalid; a further 180 end with an invalid or truncated continuation. Returning validation errors to the model is a possible extension, but further generation attempts could increase cost or repeat the same errors. Its benefit would need a separate evaluation. The no-progress rule terminates repeated failures without an intervening successful action, and resource limits bound other episodes. These termination conditions do not prove that the affected programs would otherwise run indefinitely. Some stopped episodes already satisfy the reference goal, further separating runtime status from task success.

Reasoning effects vary by method, model and metric. Gemma's pooled SR increases with reasoning in ProgPrompt and AP, while its ProgPrompt PSR and Exec decrease; CaP's pooled performance decreases for both models. AP's checkpoint conditions also respond differently. Reasoning-enabled conditions have higher median initial-plan request time for ProgPrompt and higher median episode-wide model-request time for CaP/AP. These timing scopes should not be conflated. Long reasoning traces and token-limit finishes are consistent with competition between reasoning and executable code under a fixed output budget, but do not fully explain the observed failures. Success, truncation, token use and request time must be interpreted together under the specified budgets.

Limitations. The findings are subject to the following limits:

- **Task coverage and repetition.** Each task-condition has one attempt. Suites contain 5–10 entries, and task identities recur across settings and scenes. A single-condition suite SR changes in steps of 10–20 percentage points. Pooled counts do not provide independent repeats or establish uncertainty between repeated generations.
- **Reference-based scoring.** A single annotated final graph can penalise reasonable alternative outcomes, especially for ambiguous goals. Class-name facts also omit instance-level distinctions. High preservation scores do not establish complete task achievement.
- **Environment and observations.** Evaluation uses one modified simulator and repaired reference graphs, with symbolic partial observations rather than learned perception. There are no controlled external perturbations. Results therefore do not establish robustness to moving objects, changing goals or perception errors.
- **Models, hardware and cost measurements.** Only two 4-bit quantised models are compared. They run on different GPU types, and ProgPrompt's reported timing and token usage cover its initial plan rather than the episode-wide scope used for CaP/AP. Absolute performance and latency may differ under other models, precision levels or hardware.
- **Baseline adaptation.** CaP is reimplemented for VirtualHome with our prompts, examples, observation API and runtime limits. The measurements describe this adaptation, not the original tabletop system.
- **Differences between AP and CaP.** AP changes prompts and examples, receives richer error messages, has a larger request budget and deadline, and continues after known simulator rejections. The comparison evaluates these choices together. The first-failure snapshot is a within-episode diagnostic, not a controlled recovery ablation.

The evidence supports model-assisted repair as a way to recover from some execution failures in generated code policies. In this evaluation, that capability incurs additional inference cost and remains dependent on producing valid code that can enter or resume execution.

E Proposed follow-up evaluations

The present evaluation covers one agent in VirtualHome, with partial symbolic observations, fixed task instructions and no controlled external perturbations. Future studies could first strengthen the comparisons within this setting, then investigate additional sources of uncertainty and interaction.

Repeated trials and controlled ablations. Repeated generations for matched task-condition pairs would permit estimates of variation between attempts. Comparisons could hold error feedback, request budgets, deadlines and policy prompts fixed while enabling or disabling recovery mechanisms. An independently executed no-recovery condition would provide evidence beyond the recorded first-failure snapshots. Separately ablating local recovery, helper regeneration and re-planning would help identify which repair scopes contribute to success and cost.

Controlled perturbations. Moving a target object or closing a previously opened container during execution would directly test adaptation to external changes. Perturbations should be matched across methods using specified events and trigger points, with unperturbed trials as controls. Evaluation could distinguish failures introduced by the perturbation from errors already present in the generated policy, and measure detection, recovery completion, final task success and additional model calls. Comparisons should retain each baseline's ability to use observations and any fallback behaviour expressed in its program.

Generation repair and goal evaluation. Initial and continuation policies could receive bounded feedback on syntax, validation errors or output truncation. Additional attempts should consume an explicit budget, with success and total inference cost compared against the current termination policy. This would test whether generation-stage repair helps or merely repeats errors at additional cost.

For ambiguous tasks, evaluation could include multiple acceptable outcomes or independently reviewed interpretations of the instruction. These should be specified consistently across methods, with changes to reference goals documented as a new evaluation protocol. Reference agreement and reviewed semantic task completion could then be reported separately.

Visual perception. Symbolic queries avoid learned-perception errors, but a generated checkpoint can still test an incorrect or insufficient condition. Replacing the query inputs with camera-based object detection or vision-language observations would introduce an additional source of uncertainty. Evaluation should distinguish:

1. whether the observation represents the relevant world state correctly;
2. whether the checkpoint tests the condition required for progress; and
3. whether the subsequent repair succeeds.

This would separate perception errors from predicate errors and measure their effects on task success, recovery frequency and inference cost.

Physical execution. A later extension could evaluate policies on a mobile manipulator under controlled, supervised conditions. Sensor-based checkpoints would need to account for partial action outcomes, such as an incomplete grasp, and for the relationship between high-level action results and continuous control. Measures could include failure detection, recovery completion, task success and model-call latency.

F Per-suite results

Tables 10 and 11 (ProgPrompt) and Tables 12 and 13 (Code as Policies), and Tables 14 and 15 (adaptive policies, per checkpoint placement) give SR / PSR / Exec (%) per suite and condition. A superscript counts trials with an output truncated at the token limit (for CaP and AP, in any model call of the trial). No ProgPrompt plan timed out; one CaP and 21 AP trials reached their task deadline.

	Temperature 0		Temperature 0.6		Temperature 1.0	
Suite (n)	R off	R on	R off	R on	R off	R on
test_seen (10)	20/46/71	20/36/45^{2}	20/44/61	20/45/57	20/29/41	10/17/31^{4}
test_unseen (10)	10/34/58	20/59/58^{1}	30/47/52	30/67/68^{1}	20/45/53	20/39/49^{2}
ambiguous (5)	0/4/37	0/4/36	0/12/36	0/4/48	0/17/28	0/18/50
env1 (10)	40/73/82	50/82/83	40/60/74	20/47/55^{2}	40/71/72	40/69/69^{2}
env2 (10)	40/64/81	40/72/82	40/74/74	60/74/80^{1}	50/86/85	30/52/69^{2}

Table 10: ProgPrompt with Qwen3.5-9B: SR / PSR / Exec (%) per suite.

	Temperature 0		Temperature 0.6		Temperature 1.0	
Suite (n)	R off	R on	R off	R on	R off	R on
test_seen (10)	20/44/76	0/22/38^{3}	0/35/66	0/29/55^{2}	0/34/67	20/38/62^{1}
test_unseen (10)	20/63/82	20/42/48^{3}	10/50/80	40/51/59^{2}	10/52/78	40/65/75^{1}
ambiguous (5)	0/12/36	0/16/44^{1}	0/21/44	0/16/61	0/10/51	0/13/61
env1 (10)	20/72/83	40/51/52^{4}	40/74/85	40/61/72^{2}	30/72/90	30/70/75^{1}
env2 (10)	60/79/86	10/16/26^{7}	30/73/79	50/79/76^{1}	20/57/75	40/65/81^{1}

Table 11: ProgPrompt with Gemma 4 12B: SR / PSR / Exec (%) per suite.

	Temperature 0		Temperature 0.6		Temperature 1.0	
Suite (n)	R off	R on	R off	R on	R off	R on
test_seen (10)	0/31/54	20/41/60	10/35/66	10/31/40^{2}	20/38/54	0/4/10^{5}
test_unseen (10)	10/41/84	0/42/64^{1}	10/38/69	10/23/40^{3}	10/28/58	0/31/51^{2}
ambiguous (5)	0/6/68^{1}	0/14/58^{1}	0/17/55	0/10/19^{2}	0/16/37	0/2/58
env1 (10)	30/65/72	20/55/65^{2}	30/70/77	40/58/58^{1}	10/67/81	10/59/78^{1}
env2 (10)	20/57/78	10/39/52^{1}	40/65/91	30/47/71^{1}	10/43/46	0/20/35^{3}

Table 12: Code as Policies with Qwen3.5-9B: SR / PSR / Exec (%) per suite.

	Temperature 0		Temperature 0.6		Temperature 1.0	
Suite (n)	R off	R on	R off	R on	R off	R on
test_seen (10)	0/11/4	10/10/10^{7}	10/13/19	10/25/42^{1}	20/20/27	10/14/32^{1}
test_unseen (10)	10/17/31	0/9/20^{7}	10/16/40	10/22/48^{2}	10/15/17	0/5/15^{3}
ambiguous (5)	0/9/33	0/2/20^{3}	0/12/36	0/0/25	0/0/23	0/0/43
env1 (10)	10/22/19	0/5/20^{6}	0/33/46	10/24/19^{1}	20/28/35	10/32/31
env2 (10)	20/29/35	10/10/10^{3}	10/19/23	20/34/36^{2}	20/33/35	10/17/27^{2}

Table 13: Code as Policies with Gemma 4 12B: SR / PSR / Exec (%) per suite.

	Temperature 0		Temperature 0.6		Temperature 1.0	
Suite (n)	R off	R on	R off	R on	R off	R on
Explicit checkpoints						
test_seen (10)	20/47/70[1]	20/47/58[2]	10/22/26	30/38/47[3]	20/28/27	20/33/45[1]
test_unseen (10)	20/60/75	30/73/75[2]	0/20/47	10/54/64[3]	10/31/43[1]	0/24/28[3]
ambiguous (5)	0/4/31[2]	0/17/71[3]	0/0/0	0/3/52[2]	0/0/22[2]	0/6/40[1]
env1 (10)	50/69/71[2]	50/76/80[2]	40/58/67	10/24/34[5]	10/27/32	10/20/43[3]
env2 (10)	50/68/73[2]	50/60/66[5]	30/35/46[1]	40/47/51[3]	30/50/49	30/49/57[2]
Implicit checkpoints						
test_seen (10)	30/47/62[1]	30/40/51[5]	10/40/54	30/44/51[3]	20/23/26	30/41/52[2]
test_unseen (10)	30/57/80	10/53/73[3]	40/65/73	10/53/50[1]	10/41/53	20/54/56[2]
ambiguous (5)	0/17/39[2]	0/9/79[4]	0/6/23[2]	0/14/53	0/0/9	0/2/22[1]
env1 (10)	50/69/64[1]	40/75/67[2]	40/65/73[1]	40/49/48[4]	30/60/66	20/42/37[2]
env2 (10)	50/75/75[1]	60/69/58[3]	40/61/56	40/49/48[2]	40/42/45	20/32/36[3]
Both placements						
test_seen (10)	40/53/55[2]	20/51/68[3]	20/26/27[2]	20/48/57[4]	20/25/35	10/21/21[3]
test_unseen (10)	10/44/70	30/69/71[4]	0/6/32	10/33/54[5]	0/10/23	20/45/37[3]
ambiguous (5)	0/3/43[2]	0/8/44[5]	0/11/48[1]	0/19/44	0/3/28[1]	0/0/20[1]
env1 (10)	50/80/83[1]	40/81/78[2]	30/53/56	40/66/66[1]	20/46/51	50/72/65
env2 (10)	40/66/64[1]	50/68/62[3]	60/60/51[1]	40/52/66[1]	10/43/53[1]	10/18/31[3]

Table 14: Adaptive policies with Qwen3.5-9B: SR / PSR / Exec (%) per suite, condition and checkpoint placement.

	Temperature 0		Temperature 0.6		Temperature 1.0	
Suite (n)	R off	R on	R off	R on	R off	R on
test_seen (10)	20/34/45	20/37/39[5]	30/40/43	20/28/26[7]	30/42/45	50/53/50[5]
test_unseen (10)	10/36/61	10/26/20[8]	10/34/48	10/36/60[3]	10/41/57	20/53/53[4]
ambiguous (5)	0/16/37	0/11/56[3]	0/0/36	0/21/56	0/3/34	0/5/47[1]
env1 (10)	50/58/49	20/20/17[10]	40/49/43	30/53/50[4]	60/69/60	30/52/41[5]
env2 (10)	60/74/74	30/35/34[9]	50/66/66	60/75/79[2]	60/70/74	60/72/58[2]
Implicit checkpoints						
test_seen (10)	10/16/9	30/36/29[7]	20/23/17[1]	30/42/44[6]	0/7/8	40/48/43[4]
test_unseen (10)	10/42/47	20/50/50[6]	10/47/31[1]	20/62/62[3]	0/46/43	10/47/42[3]
ambiguous (5)	0/7/50	0/3/12[5]	0/0/6	0/4/64	0/9/19	0/7/45
env1 (10)	10/26/37	50/66/48[7]	20/29/33	50/71/60	0/17/19	60/72/63[2]
env2 (10)	30/53/46	30/52/54[6]	30/48/40	60/69/68[3]	40/56/44	50/72/62[4]
Both placements						
test_seen (10)	20/24/24[1]	10/18/23[8]	20/20/18	10/20/28[5]	30/30/13	10/25/33[7]
test_unseen (10)	10/40/47	0/16/12[9]	10/41/52	10/38/49[5]	10/45/47	10/27/30[7]
ambiguous (5)	0/7/31	0/9/57[5]	0/16/35	0/12/57[1]	0/0/18	0/0/49[1]
env1 (10)	30/52/52	40/46/33[9]	20/36/33[1]	30/43/40[4]	30/49/47	30/42/39[4]
env2 (10)	50/60/60	30/50/56[6]	40/59/74	40/45/26[8]	40/55/69	40/59/44[6]

Table 15: Adaptive policies with Gemma 4 12B: SR / PSR / Exec (%) per suite, condition and checkpoint placement.

G Ambiguous-goal tasks

The five tasks of the ambiguous suite name a goal without specifying it (*make dinner*, *cook lunch*), and each is scored against one annotated final graph (Table 16). No method completed any of them: 0 of 60 ProgPrompt, 0 of 60 CaP and 0 of 180 AP trials. This appendix compares what the methods did instead, using the final states of all 300 trials and a few representative episodes.

Task and annotated goal	Cond.	PP	CaP	AP
Make dinner: salmon heated, stove on with a frying pan, salmon on a plate at the kitchen table	7	6.9	3.6	2.8
Make breakfast: bread slice heated, stove on with a frying pan, bread on a plate at the table	9	13.9	7.4	6.9
Bring some breakfast to the coffeetable: cereal on a plate at the bedroom coffee table, fork in hand	11	18.2	14.4	13.6
Collect 4 fruits . . . in the dishbowl: apple, peach and plum on the dishbowl	14	0.0	1.2	3.0
Cook lunch: salmon on a frying pan, stove on	8	22.4	10.3	9.0
PSR, all five		12.3	7.4	7.1
Exec		44.3	39.7	38.3
Mean model calls		3.9	4.1	14.1
Trials truncated		1	7	45
Trials		60	60	180

Table 16: Ambiguous-goal tasks: the annotated goal (summarised), its number of goal conditions, and PSR (%) per method, pooled over both models and all conditions (and, for AP, placements). SR is 0 for every task and method.

Recovery does not help when the goal is misread. ProgPrompt has the highest PSR on this suite (12.3%), as on the others, while AP is lowest (7.1%) despite making 3.6 times as many model calls. AP's recovery is triggered by failed checkpoints, actions and exceptions, so it repairs a plan that does not execute, not a plan that executes the wrong interpretation. The suite also strains AP's generation: its policies are longer, 45 of 180 trials had a truncated call even with the 16k-token budget, and only 39 trials ran to completion; 67 ended with invalid code, 23 made no progress and 20 reached a budget. Much of every method's partial credit is incidental: proximity relations such as the agent being close to the coffee table, which any plan that walks there satisfies.

Make dinner: three sensible dinners, none annotated. The annotation fries the salmon on the stove and serves it on a plate. AP (Qwen, implicit, temperature 0.6, reasoning on) wrote two helpers, `acquire_salmon` and `heat_salmon_in_microwave`; its `putin(salmon, microwave)` failed because the salmon was not held, a local recovery grabbed it, and the salmon was heated in the microwave, meeting 1 of 7 conditions (`salmon HEATED`). CaP with Qwen at temperature 0 did the same and then returned `False` when it could not grab a plate. The best-scoring plan of any method, ProgPrompt with Qwen at temperature 1.0 (3 of 7), put the frying pan *inside* the stove and switched the stove on, but put the salmon away in a kitchen counter uncooked: its partial credit came from the stove and pan, not from making dinner.

Bring some breakfast: recovery moves the plan towards the goal. The best result on the suite is an AP trial (Qwen, both placements, temperature 0.6, reasoning on) with 8 of 11 conditions. Its policy grabbed the cereal and tried to put it on a plate with `putin`, which fails because a plate is a surface. After repeated local recoveries, the episode picked up the plate while still holding the cereal, carried both to the bedroom and put the plate on the coffee table. It missed only `cereal ON plate` and the fork, which the instruction does not mention. ProgPrompt and CaP reached at most 5 of 11 by carrying a single item, most often a bread slice or cereal, to the table.

Collect four fruits: an annotation and a harness artefact. This task's annotation places only three fruits (apple, peach, plum) on the dishbowl. All 12 ProgPrompt plans scored 0 because they never ran: ProgPrompt uses the task name as the function name, and this name contains commas, so `def collect_4_fruits_such_as_apple,_banana,...():` is not valid Python. The plans themselves were sensible (grab four fruits, put them in the dishbowl); CaP and AP are unaffected because their entry point is always `policy()`. CaP and AP grabbed an apple or the bananas but never placed a fruit on the dishbowl: they typically used `putin` on it, and the dishbowl, like the plate, is a surface. AP's recoveries tried to open the bowl and retried `putin` for dozens of actions without switching to `putback`, until a budget, the no-progress rule or an invalid re-plan ended the episode. Recovery repairs a missing precondition (Figure 3), but not a wrong belief about which action applies.

Summary. On ambiguous goals, the methods' interpretations are often reasonable (a microwaved salmon is a dinner) but differ from the single annotated end state, and adaptive policies' execution feedback cannot tell them that. Progress on this suite needs either goal clarification, for example by asking a question before planning, or evaluation that accepts any valid end state.